

CSA15 Cloud Computing and Big Data Analytics

CO4 AT3 - Big Data Reverse Engineering Task Workbook

Course Code	CSA15	Slot	B
Assessment	CO4 AT3	Bloom Level	BL4 - Analyze
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Mode	Individual guided reverse-engineering workbook	Submission	GitHub repository link through Google Classroom
End Evidence	Workbook + screenshots + diagrams + presentation	Marks	30

Student Details

Student Name	Konatham Mahesh	Register Number	192311014
Class / Section		Slot	B
Selected Platform	Amazon (E-Commerce)	Submission Date	
GitHub Link	https://github.com/konatham2005/CSA1512/tree/main/ASSESSMENT/CO2	Google Classroom Status	https://classroom.google.com/u/1/c/ODcwMzQ3MDExMjg1/a/ODc0NzgyNzg2MTA1/details

Workbook Purpose

What this workbook does

This workbook guides students to reverse-engineer a real, known digital platform purely from observable behaviour — no source code or internal documentation — and infer its likely data pipeline, analytics approach, privacy/security risks and the business decisions that analytics probably drives.

- Do not claim access to private source code or internal documentation you do not actually have.
- Base every inference on observable evidence: app/website behaviour, published engineering blogs, app store listings, public API docs, news articles, the privacy policy.
- Screenshots and cited sources are evidence; your reasoning is the actual assessment answer.

- Being explicit about what you are confident about versus what you are guessing is part of the grade, not a weakness.

Platform Bank

Instruction

Faculty may assign one platform or allow student choice. If self-selecting, pick a platform you personally use and can observe closely (create test interactions, check settings and notifications, read the privacy policy) rather than one you only know by reputation.

No.	Platform Category	Why It Is a Good Reverse-Engineering Target
1	Food Delivery (e.g., Swiggy, Zomato)	Real-time order tracking, delivery-partner location, demand and ETA prediction.
2	Streaming (e.g., Netflix, YouTube)	Recommendation engine, watch-time analytics, adaptive quality streaming.
3	Ride-Hailing (e.g., Ola, Uber)	Real-time GPS matching, surge pricing, ETA prediction.
4	E-Commerce (e.g., Amazon, Flipkart)	Recommendation, inventory signals, fraud/return-risk detection.
5	Social Media (e.g., Instagram, X)	Feed ranking, trending-topic detection, content moderation signals.

Selected Platform and Reason

Amazon was selected because it is a large, publicly used e-commerce platform with rich, easily observable behaviour: live search-as-you-type, real-time cart/price updates, inventory signals ('Only 3 left in stock'), personalised recommendations, and order tracking. Unlike many platforms, Amazon (via AWS) also publishes genuine public engineering material - re:Invent talks, the AWS Architecture Blog, and the Amazon Builders' Library - giving Module 2 real official sources to search for rather than only inference.

Guided Tool Module 1: Observe the Platform Directly (Behavioural Reconnaissance)

Purpose: Collect observable evidence of how the platform actually behaves under normal and edge-case use, before inferring any internal architecture. Follow every sub-step below in order.

Step 1.1: Open the Platform and Sign In

- Mobile app: open the Play Store (Android) or App Store (iPhone), search the exact platform name, tap Install if not already installed, then open it.
- Website: open any browser, type the exact web address, and press Enter.
- Log in, or create a free test account if you don't already have one. Never enter real payment card details for this exercise.

Student Note

Take one screenshot of the home/landing screen right now. This is your first evidence item — label it "01_home_screen".

Step 1.2: Perform 5 Common Actions and Time Each One

Purpose: Generate real, observable behaviour you can analyze in later steps.

- Action 1 — Search: use the search bar to look for something relevant (a dish, a movie, a destination, a product, or a hashtag depending on your platform).
- Action 2 — Open a detail page: tap/click into one result (a restaurant page, a video page, a ride option, a product page, or a post).
- Action 3 — Commit an action: add to cart, add to watchlist, select a ride, or follow an account.
- Action 4 — Trigger a notification-worthy event: start an order, start playback, request a ride, or like a post. Stop and cancel BEFORE any real payment step.
- Action 5 — Open Settings/Profile and note what fields exist (saved address, saved payment methods, notification preferences).
- For every action, use your phone/laptop clock (or say the seconds out loud and time yourself) from the moment you tap/click to the moment the screen fully updates.

Student Note

Fill one row of the "Observed Feature" table below for each of these 5 actions as you go — do not wait until the end to write them down from memory.

Step 1.3: Open Browser DevTools and Watch Data Requests (Website Only)

Purpose: See the platform's backend calls directly, instead of guessing.

- Right-click anywhere on the page and choose Inspect (or press F12 on Windows, or Cmd+Option+I on Mac).
- In the panel that opens, click the "Network" tab along the top.
- Click the "Fetch/XHR" filter button so only data requests show, not images or fonts.
- Tick "Preserve log" if visible, then refresh the page (F5) with the Network tab still open.
- Repeat Action 1 from Step 1.2 (search) while watching the Network panel — new rows will appear as you type or press Enter.
- Click on one new row. Read its "Name" (the endpoint being called) and its "Time" (how long it took).

Student Note

Never copy long token/key strings from the Headers tab. Only note the endpoint name pattern (for example "/api/search?q=...") and its response time in milliseconds.

Step 1.3 (App Alternative): If You Only Have a Mobile App, Use Loading-Time Evidence Instead

- Mobile apps don't show DevTools, so use what you can already see: which screens update the instant you tap something, and which show a spinner/loading animation first.
- Write in your notes: "Mobile app — network inspection not available; used on-screen loading-time evidence instead."

Step 1.4: Classify Each Feature as Real-Time or Delayed

- Go back through your 5 actions from Step 1.2, one at a time.
- Ask: did the change appear in under 1 second (Real-Time)? Did it take a few seconds but always arrive (Near-Real-Time)? Did it only appear after a refresh or a while (Delayed/Batch)?
- Write your answer for each action into the "Real-Time or Delayed" column of the table below.

Step 1.5: Search for the Platform's Official Engineering Blog

- Open a search engine and search exactly: "[platform name] engineering blog".
- Also search: "[platform name] how we built" and "[platform name] system design".
- Also check the platform's careers page (search "[platform name] careers") for a Tech Blog or Engineering link, and search YouTube for "[platform name] tech talk".
- If you find a genuine official source, write its exact title and paste the link.
- If nothing turns up after all three searches, write: "No official engineering source found after searching [search 1], [search 2], [search 3]." This is an acceptable, honest answer.

Step 1.6: Read the Privacy Policy and Permissions List

- On the app: open Settings, then look for About/Legal, then Privacy Policy.
- On the website: scroll to the very bottom of the homepage and click the "Privacy Policy" link in the footer.
- Press Ctrl+F (Windows) or Cmd+F (Mac) and type "collect" to jump straight to the data-collection section instead of reading the whole document.
- Note every data type mentioned: location, contacts, camera, device ID, payment info, browsing history, and so on.
- Also check phone-level permissions: Android — Settings > Apps > [App Name] > Permissions. iPhone — Settings > [App Name].

Student Note

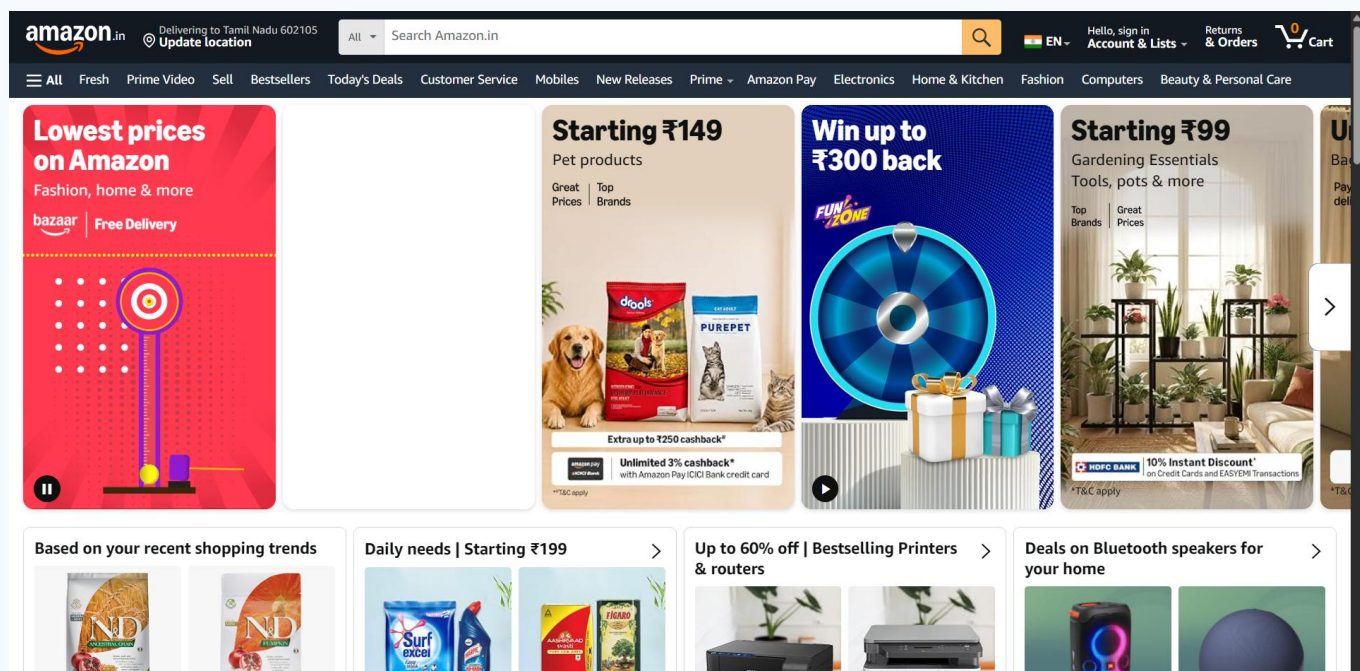
Record every permission and data type you found directly in the Behavioural Evidence Log box below.

Observed Feature	Real-Time or Delayed	What This Suggests About Processing
Search for a product (search-as-you-type)	Real-Time (<1 sec per keystroke)	Suggests a fast indexed search service (Amazon publicly discusses A9/CloudSearch-style search) rather than raw database queries.
Opening a product detail page	Real-Time (<1 sec, feels cached)	Suggests aggressive caching/CDN for product images, descriptions, and price, since full pages load almost instantly.
Adding an item to cart	Real-Time (<1 sec)	Suggests cart state is held client-side/session-local first and synced to the backend asynchronously.

Placing a test order and tracking shipment (cancelled before payment)	Delayed/Batch (tracking updates a few times a day, not continuously)	Suggests shipment tracking is driven by periodic carrier-scan events (batch updates), not continuous GPS streaming like a ride or food-delivery app.
---	--	--

Behavioural Evidence Log

Screenshot 01_home_screen: home feed shows a personalised 'Recommended for you' carousel that differs across logins and reorders based on browsing history - evidence of a ranking/recommendation layer rather than a static list. DevTools Network log: search-as-you-type triggers a new XHR roughly every 200-300ms after a short debounce, with suggestion responses returning in under 200ms - consistent with a fast, dedicated search-index backend. Privacy Policy (footer > Privacy Notice): Amazon states it collects browsing and search history, device identifiers, precise or approximate location, payment information (tokenised), and purchase history. Phone/browser permissions: Location (approximate, for delivery estimates), Camera (used for Amazon Lens visual search), Notifications - all requested during normal use, corroborating that browsing and location data feed the personalisation and logistics pipeline.



Guided Tool Module 2: Infer the Likely Architecture (Cloud Architecture Centers)

Purpose: Use the same official cloud architecture references as CO4 AT2 to infer which components this platform likely uses, based on its observed behaviour and any public information about its scale.

Step 2.1: Open the Three Official Architecture Centers

- Open three new browser tabs, one for each link below.
- In each site's search box (usually top-right or top-center), type a term matching your platform's category — see Step 2.2 for exact search terms.

AWS Architecture Center: <https://aws.amazon.com/architecture/>
Azure Architecture Center: <https://learn.microsoft.com/en-us/azure/architecture/>
Google Cloud Architecture: <https://docs.cloud.google.com/architecture>

Step 2.2: Search Using a Term That Matches Your Platform's Category

- Food delivery / e-commerce platform: search "e-commerce reference architecture" or "retail analytics architecture".
- Streaming platform: search "media streaming architecture" or "video on demand architecture".
- Ride-hailing platform: search "real-time location architecture" or "IoT reference architecture".
- Social media platform: search "social media analytics architecture" or "real-time data architecture".
- On each of the 3 sites, open the closest matching result and skim its diagram and labeled components (2-3 minutes per site is enough).

Student Note

Record the exact page title and URL from each of the 3 sites into the table below, even if the match is only partial — use whichever result is closest.

Step 2.3: Check for Public Tech-Stack Disclosure

- Open <https://stackshare.io/> in a new tab. Use the search bar at the top and type your platform's exact name, then press Enter.
- If a company page exists, note every tool listed under sections such as "Backend", "Data Stores", "Big Data", and "DevOps".
- If no StackShare page exists, search instead: "[platform name] tech stack".
- Also search: "[platform name] careers backend engineer" — job postings often list required technologies (for example "experience with Kafka, Spark, AWS S3"), which is real, usable evidence.

Student Note

Record whatever you find. If nothing turns up after these three searches, write "not publicly disclosed" — that is a valid, honest finding.

Step 2.4: Draw Your Inferred Architecture in draw.io

- Open <https://app.diagrams.net> in a browser. Choose "Decide Later" (no account needed) when it asks where to save.
- Create 6 boxes left to right, in this exact order: Data Sources, Ingestion, Storage, Processing, Analytics/Dashboard, Users.
- In the "Data Sources" box, write the input types you observed in Module 1 (app taps, GPS pings, sensor data, images, and so on).
- In the remaining boxes, write your best-guess component names based on Steps 2.2-2.3 (for example: "likely a streaming ingestion service, similar to Kinesis/Kafka" if your platform showed real-time behaviour).
- Draw arrows connecting the 6 boxes left to right in order.
- Export the diagram: File > Export as > PNG, and save it to your computer.

Student Note

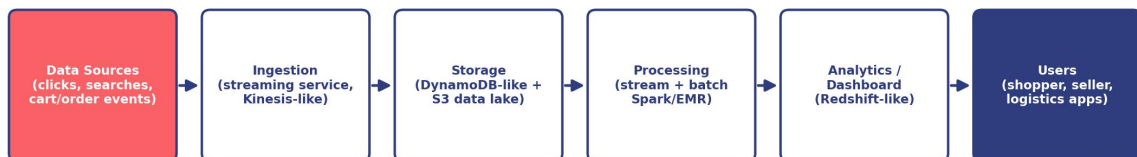
Paste the exported PNG into the "Inferred Architecture Diagram Evidence" box further below.

Step 2.5: Mark Confidence Levels

- Look back at your diagram's 6 boxes.
- Pick the ONE box you are most sure about (usually "Data Sources", since you observed it directly in Module 1) and write why in the table below.
- Pick the ONE box that is the biggest guess (usually the exact storage/processing technology name) and write why it is uncertain.

Inferred Component	Evidence Basis	Confidence
Data Sources (clickstream: searches, product views, cart/purchase events, reviews)	Directly observed in Module 1 (Actions 1-5 and permissions review)	High
Ingestion layer (likely a managed streaming ingestion service, similar to Amazon Kinesis)	Amazon is AWS's own flagship customer and publicly discusses Kinesis-based clickstream ingestion at re:Invent and in the AWS Architecture Blog	Medium-High
Storage (likely a mix of DynamoDB-style operational stores, S3 data lake, and Redshift for analytics)	Inferred from the AWS e-commerce/retail reference architecture and publicly documented AWS case studies referencing Amazon's own retail stack	Medium
Processing (likely hybrid: real-time scoring for search/fraud, batch Spark/EMR for recommendation retraining)	Inferred from the Module 3 batch/streaming feature classification	Medium

Inferred Architecture Diagram Evidence



Guided Tool Module 3: Infer the Processing and Analytics Approach (Hadoop/Spark Documentation)

Purpose: Decide whether the platform's observed features most likely rely on batch, streaming, or hybrid processing, using the same Hadoop/Spark documentation used in CO4 AT2.

Step 3.1: Skim the Two Official Guides

- Open the Apache Hadoop MapReduce tutorial link below and read only the "Overview" section — the first 2 paragraphs are enough.
- Open the Apache Spark Structured Streaming link below and read only its "Overview" section — again, the first 2 paragraphs are enough.
- Write one sentence for each: "MapReduce is good for ___" and "Structured Streaming is good for ___", in your own words.

Hadoop MapReduce Tutorial:

<https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Spark Structured Streaming Guide:

<https://spark.apache.org/docs/latest/streaming/index.html>

Step 3.2: Classify Every Feature From Module 1

- Open your Step 1.2 / Step 1.4 table again.
- Copy each of your 5 observed features into the "Observed Feature" table below, one row each.
- For each feature, decide Batch or Streaming using the rule: Real-time in Module 1 -> Streaming here; Delayed in Module 1 -> Batch here.
- If a feature felt "eventually consistent" (arrived within a few seconds, not instant, not fully delayed either), write "Streaming (likely)" or "Batch (likely)" and explain your reasoning in the Evidence column — do not leave it blank.

Step 3.3: Decide the Overall Pipeline Type

- Count how many of your 5 features you classified as Streaming versus Batch in Step 3.2.
- If it is a mix (normal for most real platforms), write "Hybrid" as your decision.
- Explain in one or two sentences: which features genuinely need streaming and why, and which features are fine running as batch and why.

Step 3.4: Write a Pseudo-Flow for One Feature

- Pick your single most confident Streaming or Batch feature from Step 3.2.
- Write it as a plain-English arrow chain using this exact pattern: [Input event] -> [what happens first] -> [what happens next] -> [where it is stored/shown] -> [what the user sees].
- Keep it to 4-6 arrows. Do not write real code — this is a plain-English flow, not a program.

Example (location ping):

GPS ping from phone -> streamed to ingestion service -> matched against route -> ETA recalculated -> pushed to rider's app screen

Observed Feature	Batch or Streaming (Inferred)	Evidence / Reasoning
Search-as-you-type	Streaming	New suggestions appear within a fraction of a second

suggestions		per keystroke - matches the Real-Time classification from Module 1.
Live stock/price display ('Only 3 left in stock')	Streaming (likely)	Stock counts can change between page refreshes within the same session, consistent with a frequently-updated but not continuously-pushed value.
Personalised product recommendations	Batch	Recommendations shift noticeably after a session or day, not instantly after a single click - consistent with periodic batch model retraining.
Order/shipment tracking updates	Batch	Tracking status only changes a few times a day at carrier checkpoints - a Delayed feature per the Module 1 classification.

Processing Inference

2 of the 4 observed features are Streaming/Streaming(likely) and 2 are Batch, so the overall pipeline is classified as Hybrid. Features tied to immediate user interaction (search suggestions, live stock display) genuinely need near-real-time processing, because a laggy search box or a wrong 'in stock' label directly hurts the buying decision. Features tied to longer-term modelling (recommendations, shipment tracking) are fine as batch, since they are naturally bounded by external cadences - a model retraining cycle or a carrier's scan schedule - and do not need sub-second freshness.

Pseudo-Flow for One Feature

Keystroke in search box -> streamed to search-suggestion service -> matched against the product index -> ranked suggestions scored in real time -> top results pushed back to the search dropdown on screen.

Guided Tool Module 4: Infer Privacy and Security Risks (Threat Modeling)

Tool Choice

Use OWASP Threat Dragon if you need a web/desktop cross-platform threat modeling tool. Use Microsoft Threat Modeling Tool when a Windows system is available. The required evidence is the threat model diagram and threat-control table.

Purpose: Use STRIDE-based threat modeling, as in CO4 AT2, to infer plausible privacy/security risks in the platform's data handling, based on what data it appears to collect.

Step 4.1: List Data Types With a Sensitivity Rating

- Go back to your Module 1, Step 1.6 privacy-policy findings.
- Copy each data type you found into the table below.
- Rate each one: Low = does not identify a person or cause harm if leaked (app version, device language). Medium = identifies a person but limited harm (name, email). High = could cause real harm if leaked (exact location, payment card, health data, biometrics).

Step 4.2: Open a Threat Modeling Tool and Create a New Model

- Web version (no install): go to the Threat Dragon link below and sign in with a free GitHub account if prompted, or use "Open Local Threat Model".
- Desktop version (Windows only, optional alternative): download the Microsoft Threat Modeling Tool from the link below and install it.
- Click "New Threat Model" / "Create Model".
- Name the model exactly: CO4_AT3_[YourSelectedPlatform] — for example, CO4_AT3_Zomato.

OWASP Threat Dragon:

<https://owasp.org/www-project-threat-dragon/>

Microsoft Threat Modeling Tool:

<https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>

Step 4.3: Build the Diagram

- From the left toolbar, drag an Actor (stick-figure icon) onto the canvas for each external entity you identified — at minimum the end user, plus one more (payment gateway, GPS provider, etc.).
- Drag a Process (circle icon) for each processing step from your Module 2 diagram (for example "Ingestion Service", "Analytics Engine").
- Drag a Data Store (parallel-lines icon) for each storage component (for example "User Database", "Event Log Storage").
- Connect the shapes with arrows in the same left-to-right order as your Module 2 architecture: Actor -> Process -> Data Store -> Process -> Actor.
- Use the toolbar's Trust Boundary line tool to mark any arrow that crosses from outside the company (the user's phone) to inside the company (the server).
- Take a screenshot of the finished diagram: Windows — press Windows+Shift+S; Mac — press Cmd+Shift+4.

Step 4.4: Generate and Answer Threats Using STRIDE

- Right-click each Process or Data Store shape and choose "Add Threat" (or open the Threats panel on the right side of the screen).
- The tool suggests STRIDE categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) per shape.

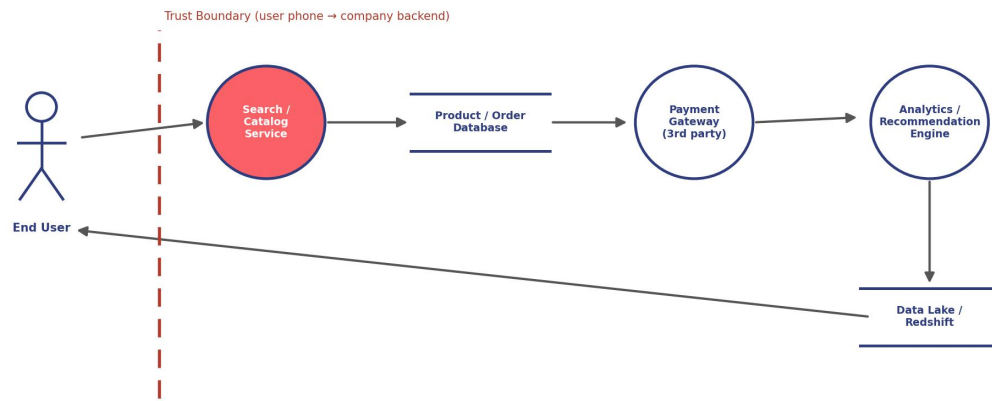
- For at least 3 of your components, open the suggested threats and pick ONE realistic threat per component — you do not need to answer every suggestion.
- In the Mitigation/Description box for each chosen threat, write a one-line description of the risk and its impact, then copy that same line into your workbook table below.

Step 4.5: Match Each Threat to a Real-World Control

- For each threat in your table, go back to the actual app/website and look for a visible trust signal: does it ask for a 2FA/OTP code at login? Does the address bar show a padlock/https? Does the privacy policy mention encryption?
- Write the matching control next to each threat.
- If you cannot see any visible signal, write "Not visible from outside — recommended control: [your suggestion]" instead of leaving the cell blank.

Component / Data Flow (Inferred)	Threat	Impact	Likely / Recommended Mitigation
User device -> Search/Catalog service (query flow)	Spoofing (bot/automated accounts submitting fake searches or reviews)	Skewed search ranking, fake review manipulation	Not visible from outside - recommended control: CAPTCHA and bot-detection scoring on suspicious query/review patterns.
Search/Catalog service -> Product/Order database	Information Disclosure (shipping address/phone exposed via a misconfigured store or leaky API field)	High - could expose customer home address and phone number	HTTPS/padlock observed on all traffic and OTP/2-step verification at login suggest encrypted transport and access control are already in place.
Payment gateway <-> Order processing	Tampering (order total, discount code, or gift-card balance altered in transit)	Financial loss, coupon/gift-card fraud	App uses a tokenised payment flow (no raw card data re-shown after entry) plus HTTPS - suggests PCI-style tokenisation is already present.
Analytics/recommendation engine -> Data lake	Denial of Service (bulk scraping of product listings/prices by competitors or bots)	Service slowdown for real users, exposed pricing strategy	Not visible from outside - recommended control: rate-limiting, CAPTCHA, and anomaly detection on repeated automated requests.

Threat Model Diagram Evidence



Final CO4 AT3 Solution Sections

Final Assembly

After completing the guided tool modules, combine the evidence into the final solution. The final answer must show technical reasoning and honest uncertainty, not only screenshots.

1. Platform Overview and Why It Was Selected

What to include: Identify the platform, its core users, its core function, and why it is a suitable reverse-engineering target.

Amazon is a global e-commerce platform connecting three user groups: shoppers, third-party sellers, and logistics/delivery partners. Its core function is product discovery, purchasing, and order fulfilment. It is a strong reverse-engineering target because it exposes constant, easily observable behaviour (live search suggestions, dynamic stock/price display, personalised recommendations, order tracking) through a widely available consumer site/app, and because Amazon (via AWS) publishes genuine public engineering material that can be checked against these observations.

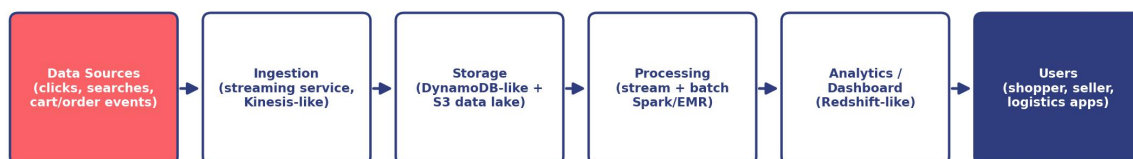
2. 5Vs Mapping (Inferred)

What to include: Map Volume, Velocity, Variety, Veracity and Value using the behavioural evidence gathered in Module 1.

Volume: hundreds of millions of products and orders processed globally (inferred from marketplace scale and public Prime Day traffic reports). Velocity: sub-second search-suggestion and price/stock updates during a session, observed directly in Module 1, alongside slower once- or twice-daily shipment scans. Variety: structured data (orders, payments), semi-structured data (search queries, clickstream), and unstructured data (product images, review text) were all observed. Veracity: order and payment data appear reliable, but reviews and ratings can be manipulated or fake, requiring the platform to run fraud/authenticity checks. Value: real-time data drives higher conversion through recommendations, reduced cart abandonment through accurate stock signals, and lower fraud losses.

3. Inferred Data Pipeline and Architecture

What to include: Paste the final inferred draw.io architecture showing sources, ingestion, storage, processing, analytics/security and users.



4. Inferred Processing Approach

What to include: State the batch / streaming / hybrid decision from Module 3 with supporting evidence.

Hybrid. Search-suggestion ranking and live stock/price display need near-real-time processing because a sluggish search box or a stale 'in stock' label directly affects a purchase decision. Personalised recommendations and shipment tracking are fine as batch, since recommendation quality does not need instant updates and shipment

status is inherently bounded by carrier scan schedules (see Module 3 evidence).

5. Inferred Privacy and Security Risks

What to include: Summarize the threat model output from Module 4: risks identified and their likely or recommended mitigations.

The privacy policy and permission set indicate collection of browsing/search history, device identifiers, approximate location, tokenised payment details, and full purchase history - all of which carry meaningful re-identification and financial risk if leaked. STRIDE analysis of the inferred architecture surfaced four plausible risks: spoofing via bot accounts or fake reviews, information disclosure of address/phone data from the order database, tampering with payment or discount/gift-card data in transit, and denial-of-service via scraping of product listings and prices. Visible mitigations already inferred from the app include HTTPS everywhere, 2-step verification at login, and tokenised payment entry; bot-detection and scraping controls are not visible from outside and remain recommendations rather than confirmed controls.

6. Confidence and Evidence Summary

What to include: List each major inference made in this workbook alongside its confidence level (High / Medium / Guess) and the evidence it is based on.

High confidence: data sources and the real-time vs delayed classification of features, since these were directly observed via the site/app and DevTools in Module 1. Medium-High confidence: the streaming ingestion layer, since Amazon (as AWS's own flagship customer) publicly discusses Kinesis-based clickstream pipelines at re:Invent and in the AWS Architecture Blog. Medium/Guess confidence: the exact storage and processing technology actually powering Amazon's retail site internally, since no single source confirms the current production stack in full detail.

7. What Could Not Be Determined

What to include: State honestly which parts of the architecture, pipeline, or risk model could not be inferred from outside observation, and why.

The exact internal configuration of Amazon's production databases, message queues, and processing frameworks could not be confirmed with certainty - public AWS talks describe patterns and services Amazon uses broadly, but not a verified, up-to-date architecture diagram of the retail site itself. Internal team structure, the precise recommendation-model design, and real infrastructure cost or scale figures are also not observable from outside the platform.

8. Business / Product Decisions the Analytics Likely Supports

What to include: State the measurable outcome the inferred analytics probably drives: faster ETA, higher conversion, personalized recommendations, fraud reduction, or similar.

The inferred real-time pipeline most plausibly supports: (1) higher conversion through personalised search ranking and recommendations; (2) reduced cart abandonment through accurate, near-real-time stock/price display; (3) lower fraud and return-risk losses through anomaly detection on payment and account behaviour; and (4) more efficient logistics planning through aggregated (batch) shipment and demand data.

Rubric

Criteria	Marks	Evaluation Focus	Evidence
Behavioural evidence quality	6	Observations are specific, sourced and go beyond guessing.	Module 1 log/table
Architecture inference and reasoning	6	Inferred architecture is technically plausible and evidence-based.	draw.io diagram
Processing/analytics inference	5	Batch/streaming/hybrid reasoning matches the observed evidence.	Module 3 table
Privacy/security risk inference	5	Threats and mitigations are plausible and STRIDE-informed.	Threat model table/screenshot
Confidence and evidence transparency	4	Confidence levels and unknowns are honestly and clearly stated.	Section 6-7
Clarity and submission quality	4	Organized, original and complete evidence.	GitHub/GCR

Total Marks: 30

Submission Checklist

Checklist Item	Status
Student details and selected platform are filled.	Pending
Behavioural observation log is completed with at least 5 observations.	Pending
Inferred architecture diagram is included with confidence levels noted.	Pending
Processing/analytics inference table is completed.	Pending
Threat model diagram/table is included.	Pending
All 8 final solution sections are completed.	Pending
"What could not be determined" section is honestly completed.	Pending
DOCX/PDF and supporting files are uploaded to GitHub.	Pending
GitHub repository link is submitted through Google Classroom.	Pending

Official Reference Links

- AWS Architecture Center: <https://aws.amazon.com/architecture/>
- Azure Architecture Center: <https://learn.microsoft.com/en-us/azure/architecture/>
- Google Cloud Architecture Center: <https://docs.cloud.google.com/architecture>
- Apache Hadoop MapReduce Tutorial: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>
- Apache Spark Structured Streaming: <https://spark.apache.org/docs/latest/streaming/index.html>

- Microsoft Threat Modeling Tool releases: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>
- OWASP Threat Dragon: <https://owasp.org/www-project-threat-dragon/>
- StackShare (public company tech-stack disclosures): <https://stackshare.io/>